

Databázová Aplikace

Projekt č. 3

Kryštof Troják

17. prosince 2025

Obsah

1	Úvod a účel aplikace	3
1.1	Použité technologie	3
2	Analýza a Funkční specifikace	4
2.1	Model případů užití (Use Case Diagram)	4
2.2	Splnění požadavků zadání	4
3	Databázový model a logika serveru	6
3.1	Diagram tříd (Class Diagram)	6
3.2	ER Diagram	7
3.3	Triggery	8
3.4	Uložené procedury (Stored Procedures)	8
4	Architektura systému	9
4.1	Projektová struktura	9
4.2	Logika transakcí	9
4.3	Implementační detaily	10
5	Uživatelská příručka a funkcionalita	11
5.1	Uživatelské rozhraní (UI/UX)	11
5.2	Autentizace a Bezpečnost	11
5.3	Dynamické filtrování dat	12
5.4	Správa souborů	12
6	Bezpečnostní prvky	13
6.1	Prevence SQL Injection (SQLi)	13
6.2	Šifrování dat	13
7	Zálohování a bnova	14
7.1	Zálohování a Obnova (Backup & Restore)	14
8	Instalace a konfigurace	14
8.1	Konfigurace (.env)	14
9	Možná budoucí rozšíření	14
10	Závěr	14

1 Úvod a účel aplikace

Dokument popisuje implementaci databázové aplikace pro správu internetového obchodu. Projekt byl vytvořen jako součást předmětu BDS (Databázové systémy). Cílem bylo navrhnout a implementovat informační systém s využitím relační databáze PostgreSQL a programovacího jazyka Python.

Aplikace slouží jako nástroj pro správu e-shopu. Umožňuje evidenci produktů včetně variant, správu zákazníků a jejich citlivých údajů, procesování objednávek a řízení přístupových práv zaměstnanců. Důraz byl kladen na bezpečnostní aspekty, jako je ochrana proti SQL Injection, bezpečné ukládání hesel pomocí moderního algoritmu Argon2 a šifrování dat na úrovni databáze.

1.1 Použité technologie

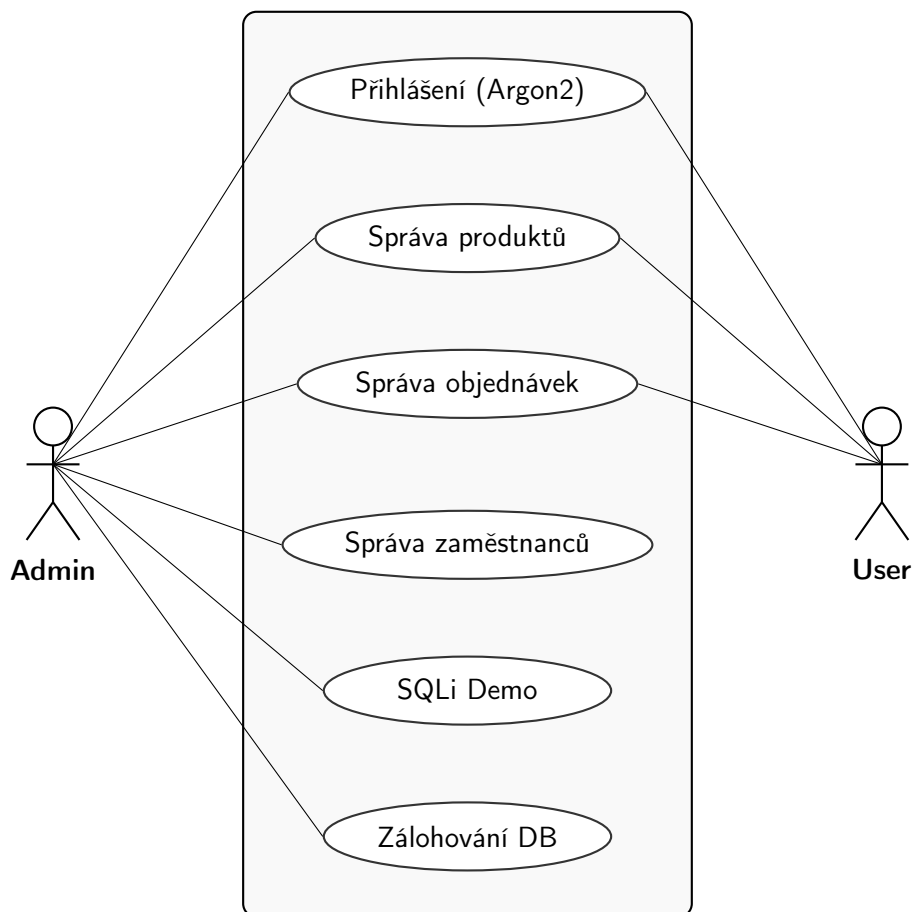
Aplikace je postavena na moderním a ověřeném technologickém stacku:

- **Backend:** Python 3.10+, Flask 3.x (Lehký, modulární webový framework).
- **Databáze:** PostgreSQL 15+ (včetně rozšíření `pgcrypto` pro šifrování).
- **ORM:** SQLAlchemy 2.0 (Objektově-relační mapování pro bezpečnou práci s daty).
- **Bezpečnost:** Argon2-cffi (Hashing hesel), Flask-Login (Správa relací).
- **Frontend:** HTML5, Bootstrap 5, Jinja2 šablony, jQuery (AJAX).

2 Analýza a Funkční specifikace

2.1 Model případů užití (Use Case Diagram)

Diagram znázorňuje interakci uživatelů se systémem. Rozlišujeme dvě role: **Zaměstnanec (User)** a **Administrátor (Admin)**. Admin dědí práva běžného uživatele a má navíc přístup k citlivým modulům.



Obrázek 1: Use Case Diagram aplikace E-shop

2.2 Splnění požadavků zadání

Tato sekce mapuje implementaci na konkrétní body zadání.

Požadavek zadání	Implementace v projektu
Desktop app (Java/Python) nebo Web framework	Aplikace je implementována v jazyce Python s využitím webového frameworku Flask .
Spustitelnost z příkazové řádky	Aplikace se spouští příkazem <code>python run.py</code> . Knihovny jsou zapsány v <code>requirements.txt</code> .
Hashování hesel (Argon2)	Využita knihovna Argon2 (<code>argon2-cffi</code>) pro bezpečné ukládání hesel. Odolné proti GPU útokům.
Validace přihlášení proti DB	Reálné ověření údajů v tabulce <code>staff_member</code> . Nejedná se o simulaci. Chybové hlášky odpovídají zadání.
Role v databázi	Aplikace se připojuje pod dedikovaným uživatelem <code>bds-app</code> , který má grantovaná práva pouze na schéma <code>bds</code> . Tabulka <code>roles</code> definuje aplikační role (ADMIN, USER).

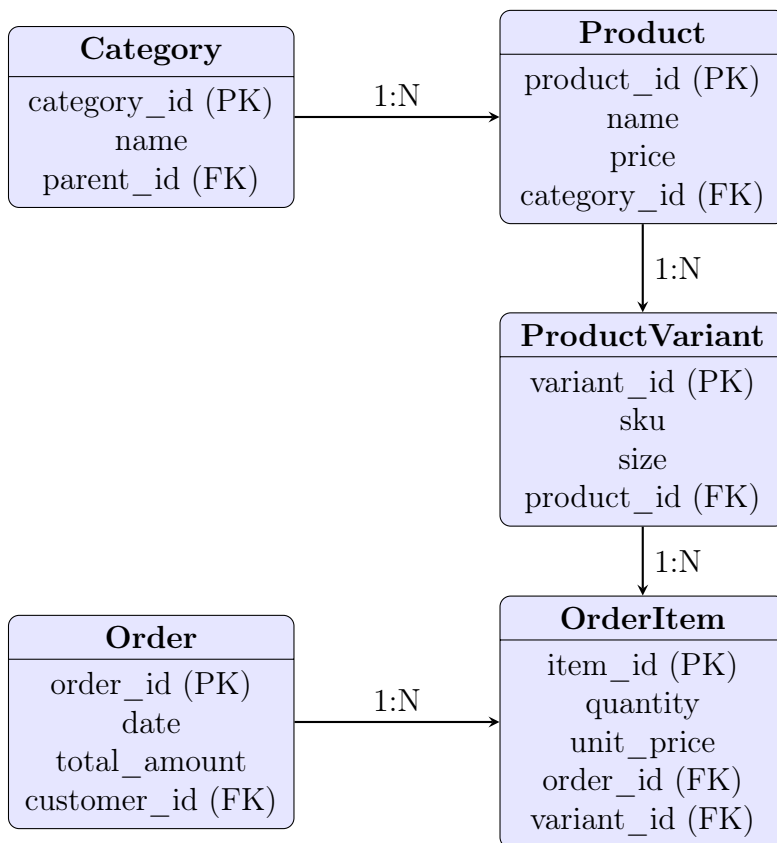
Požadavek zadání	Implementace v projektu
Oddělené schéma	Veškeré tabulky jsou umístěny ve schématu <code>bds</code> , nikoliv <code>public</code> .
CRUD operace (min. 5 entit)	Plné CRUD pro: Produkty, Kategorie, Objednávky, Zaměstnanci, Zákazníci . FindAll implementováno pro všechny.
Detailní pohled (JOIN)	Detail produktu (JOIN varianty/obrázky), Detail objednávky (JOIN zákazník/položky), Detail zákazníka (JOIN adresy).
Transakce (více dotazů)	Vytvoření objednávky je atomické. Vkládá se hlavička i položky. Chyba způsobí <code>ROLLBACK</code> .
5+ GUI oken	Implementovány samostatné pohledy (šablony) pro seznamy, formuláře a detaily každé z 5 entit.
Filtrování dat	Seznamy produktů i objednávek umožňují vyhledávání (Full-text) a filtrování dle kategorií nebo statusů.
SQL Injection Demo	Stránka <code>/sql-injection-demo</code> ukazuje zranitelný vs. bezpečný kód na speciální tabulce <code>bds.dummy_sql_i</code> .
Logování výjimek	Použit <code>TimedRotatingFileHandler</code> pro logování chyb do složky <code>logs/</code> .
Zálohování (Backup)	Skript <code>backup_db.sh</code> provádí <code>pg_dump</code> a simulovaný upload na S3 Cloud Storage.
Obnova (Restore)	Skript <code>restore_db.sh</code> stáhne zálohu a provede <code>pg_restore</code> .
Šifrovaná komunikace (SSL)	Aplikace běží na HTTPS (port 443). Využívá se kontext <code>adhoc</code> (Flask) nebo self-signed certifikáty generované OpenSSL.
Externí konfigurace	Citlivé údaje (DB hesla, Secret Key) jsou v souboru <code>.env</code> , nejsou v kódu.
Seznam licencí	Soubor <code>LICENSES.md</code> obsahuje seznam použitých knihoven a jejich licencí (MIT, BSD, Apache).
GitLab (25 commits, 5 MR)	Projekt je verzován v Gitu, obsahuje požadovaný počet commitů a merge requestů.
Encryption at Rest (TDE)	Implementováno šifrování citlivých sloupců (<code>phone_number</code>) pomocí <code>pgcrypto</code> .
Unit Testy	Implementována sada automatických testů nad in-memory databází.
Stored Procedures	Využití uložených procedur pro aktualizaci stavu plateb.

3 Databázový model a logika serveru

Jádrem aplikace je databáze navržená ve 3. normální formě (3NF).

3.1 Diagram tříd (Class Diagram)

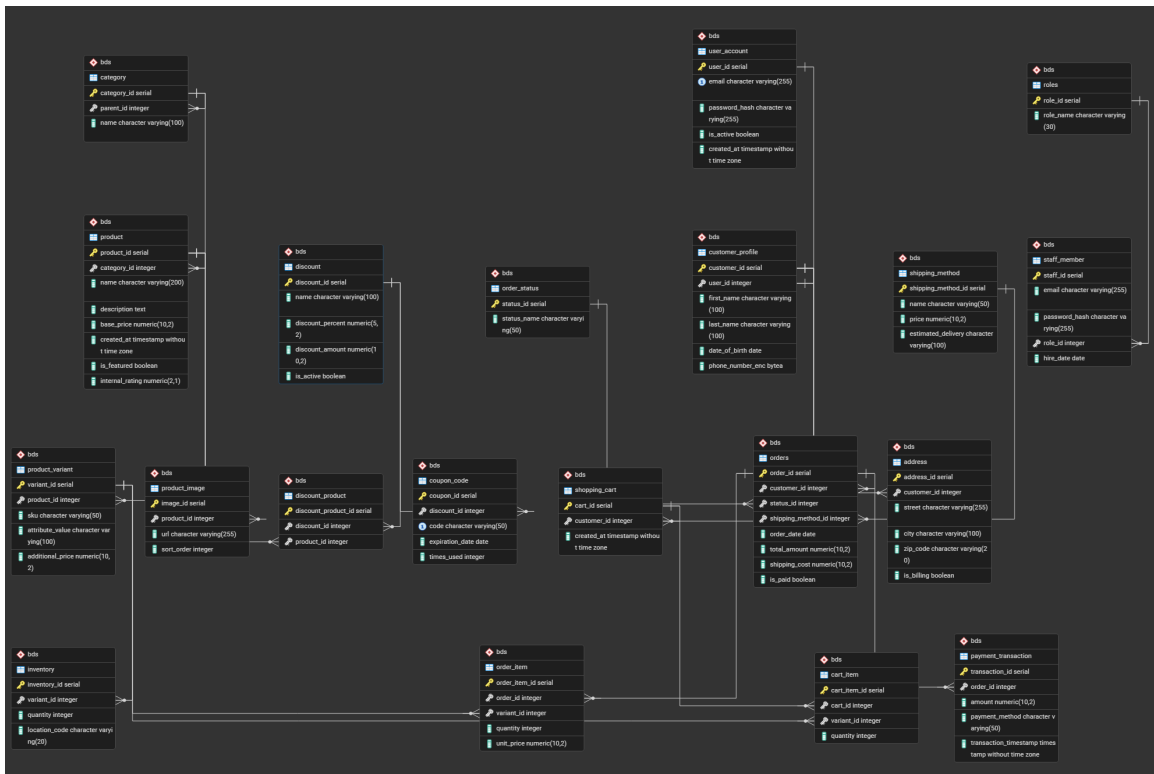
Diagram níže vizualizuje databázové modely definované v `app/models.py` pomocí ORM SQLAlchemy. Ukazuje atributy a vazby mezi entitami (1:N, M:N).



Obrázek 2: Diagram tříd datového modelu

3.2 ER Diagram

Logická struktura databáze zahrnuje entity pro produkty, varianty, sklad, zákazníky, adresy, objednávky a platby.



Obrázek 3: Entitně-relační diagram (ERD) databáze e-shopu

3.3 Triggery

Pro zajištění konzistence dat a přenesení výpočetní logiky blíže k datům byl v databázi implementován Trigger. Tento mechanismus automaticky přepočítává celkovou cenu objednávky kdykoliv, kdy dojde k přidání nové položky.

PL/pgSQL Trigger (sql_scripts/ddl_schema.sql)

```
1 CREATE OR REPLACE FUNCTION update_order_total()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     UPDATE orders
5     SET total_amount = (
6         SELECT COALESCE(SUM(quantity * unit_price), 0)
7         FROM order_item
8         WHERE order_id = NEW.order_id
9     )
10    WHERE order_id = NEW.order_id;
11    RETURN NEW;
12 END;
13 $$ LANGUAGE plpgsql;
14
15 -- Trigger se spouští po vložení položky do košíku
16 CREATE TRIGGER trg_after_order_item_insert
17 AFTER INSERT ON order_item
18 FOR EACH ROW
19 EXECUTE FUNCTION update_order_total();
```

3.4 Uložené procedury (Stored Procedures)

Kromě triggerů aplikace využívá i uložené procedury pro zapouzdření složitější logiky na straně databáze. Příkladem je procedura `update_order_paid_status`, která atomicky aktualizuje stav zaplacení objednávky na základě transakce.

Uložená procedura (sql_scripts/ddl_schema.sql)

```
1 CREATE OR REPLACE PROCEDURE update_order_paid_status(trans_id INT)
2 LANGUAGE SQL
3 AS $$
4     UPDATE orders
5     SET is_paid = TRUE
6     WHERE order_id = (
7         SELECT order_id FROM payment_transaction
8         WHERE transaction_id = trans_id
9     ) AND is_paid = FALSE;
10 $$;
```


4 Architektura systému

Projekt je postaven na tzv. monolitické architektuře. To znamená, že projekt je sestaven do jednoho velkého spustitelného celku. Tuto strukturu jsem hlavně zvolil z důvodu efektivity vývoje a snížení complexity nasazení. Jelikož je projekt zaměřen primárně na databázovou vrstvu, transakční zpracování a bezpečnost, monolitický přístup umožňuje přímou práci s datovými modely (pomocí ORM SQLAlchemy) bez nutnosti vytvářet a udržovat oddělenou API vrstvu pro frontend. Využití Server-Side Renderingu (Jinja2), zjednodušuje správu konzistence dat a usnadňuje nasazení aplikace jako jedné služby. Projekt dále využívá architekturu MVC (Model-View-Controller). Ve frameworku Flask je tato architektura realizována pomocí tzv. **Blueprints**.

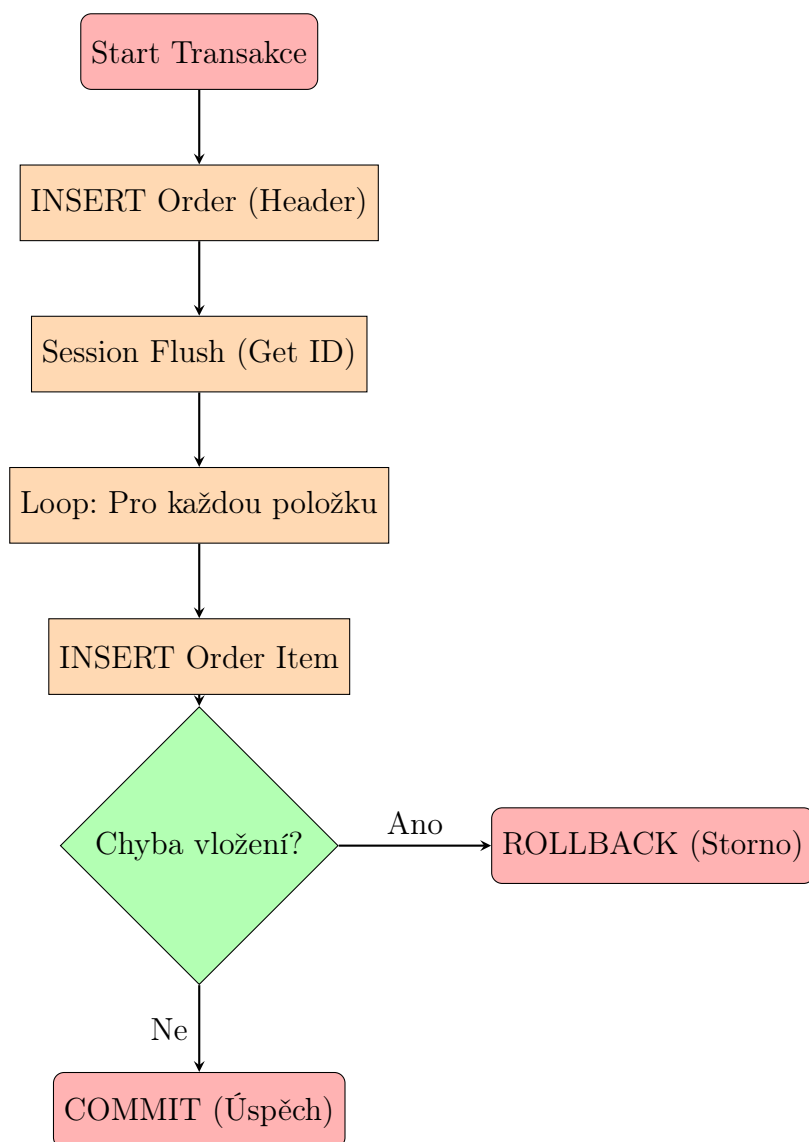
4.1 Projektová struktura

Adresářová struktura

```
/project-root
app/
  __init__.py      # Inicializace aplikace, DB a loggeru
  models.py        # Definice DB schématu (SQLAlchemy)
  utils.py         # Pomocné funkce (upload souborů)
  blueprints/      # Logika aplikace (Controllers)
    auth.py        # Autentizace (Login/Logout)
    products.py    # Správa produktů
    orders.py      # Komplexní logika objednávek
    staff.py       # Administrace zaměstnanců
  templates/       # HTML šablony (Views - Jinja2)
tests/             # Automatické testy
scripts/           # Bash skripty pro správu
run.py            # Vstupní bod aplikace
```

4.2 Logika transakcí

Nejsložitější proces v aplikaci je vytvoření objednávky. V aplikaci využíváme metodu `flush()`, která vygeneruje primární klíč (`order_id`) pro hlavičku objednávky ještě před potvrzením transakce. To umožňuje relačně propojit položky objednávky v rámci jedné atomické operace.



Obrázek 4: Flowchart transakčního zpracování objednávky

4.3 Implementační detaily

Aplikace využívá vlastní dekorátor `@admin_required`, který zajišťuje, že k určitým routám (např. správa zaměstnanců) mají přístup pouze uživatelé s rolí ADMIN.

Dekorátor pro kontrolu oprávnění (app/blueprints/staff.py)

```

1 def admin_required(f):
2     @wraps(f)
3     def decorated_function(*args, **kwargs):
4         # Kontrola, zda je uživatel ADMIN
5         if not current_user.is_authenticated or \
6            current_user.role.role_name != 'ADMIN':
7             flash('Tato sekce je pouze pro administrátory.', 'danger')
8             return redirect(url_for('index'))
9         return f(*args, **kwargs)
10    return decorated_function
  
```

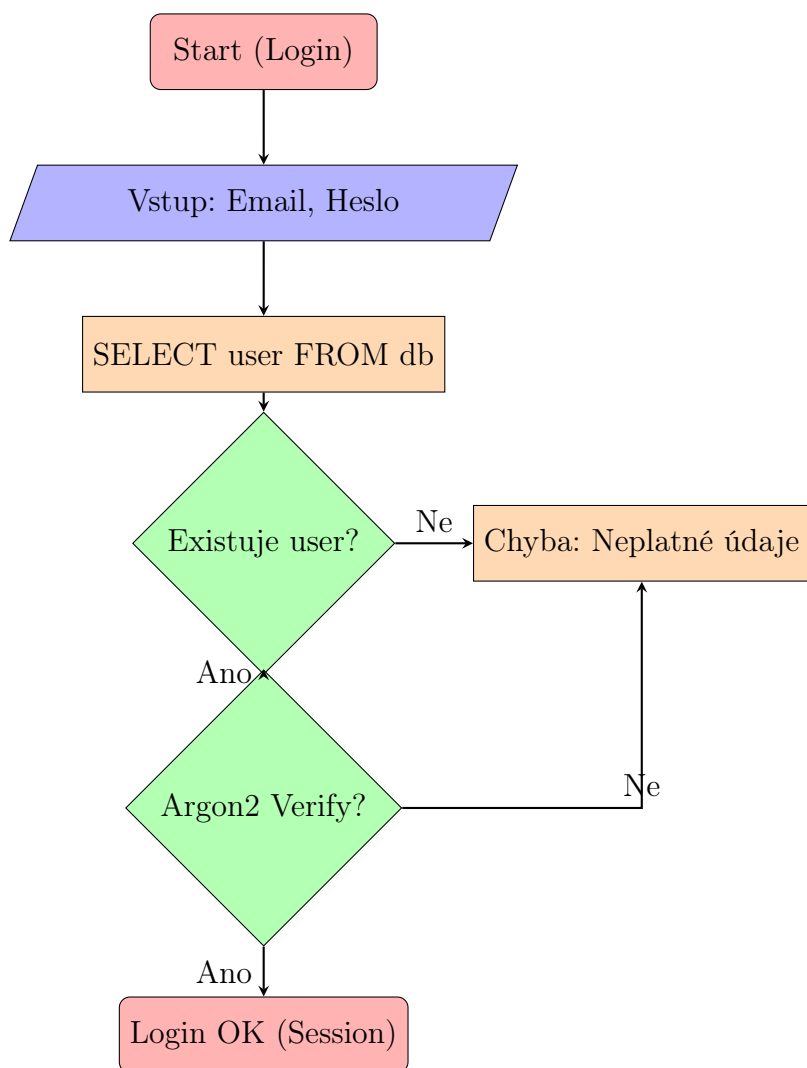
5 Uživatelská příručka a funkcionalita

5.1 Uživatelské rozhraní (UI/UX)

Frontend aplikace je postaven na frameworku Bootstrap 5, což zajišťuje plnou responzivitu. Aplikace je tak plně ovladatelná z desktopu a případně by šlo jednoduše vytvořit verzi pro telefony. Pro interaktivní prvky (modální okna, dynamické vyhledávání) je využita knihovna jQuery.

5.2 Autentizace a Bezpečnost

Proces přihlašování je kritický bod aplikace. Diagram níže zobrazuje logiku ověřování uživatele a jeho hesla pomocí algoritmu Argon2.



Obrázek 5: Flowchart přihlašovacího procesu

5.3 Dynamické filtrování dat

Seznamy v aplikaci (Produkty, Objednávky) podporují pokročilé filtrování. Uživatel může kombinovat textové vyhledávání s výběrem kategorie nebo statusu.

Filtrování produktů (app/blueprints/products.py)

```
1 query = Product.query.join(Category, isouter=True)
2
3 # Fulltextové vyhledávání v názvu, popisu a ID
4 if search_term:
5     conditions = [
6         Product.name.ilike(f'"{search_term}"'),
7         Product.description.ilike(f'"{search_term}"'),
8         cast(Product.product_id, String).ilike(f'"{search_term}"')
9     ]
10    query = query.filter(or_(*conditions))
11
12 # Filtrace dle kategorie
13 if category_id:
14     query = query.filter(Product.category_id == category_id)
```

5.4 Správa souborů

Pro nahrávání obrázků k produktům byla vytvořena pomocná funkce v `utils.py`, která zajišťuje bezpečné uložení souboru na disk a vygenerování unikátního názvu.

Upload obrázku (app/utils.py)

```
1 def handle_image_upload(product_id, file):
2     if not file or not file.filename:
3         return None
4     # Zabezpečení názvu souboru
5     original_filename = secure_filename(file.filename)
6     filename = f"{product_id}_{original_filename}"
7
8     # ... uložení na disk ...
9     file.save(file_path)
10
11     return url_for('static', filename=f'uploads/products/{filename}')
```

6 Bezpečnostní prvky

6.1 Prevence SQL Injection (SQLi)

Aplikace striktně využívá Prepared Statements. Pro edukační účely obsahuje modul `sql_i.py` s ukázkou zranitelného kódu na tabulce `bds.dummy_sql_i`, kde je vidět nebezpečí spojování řetězců.

Zranitelný vs. Bezpečný dotaz (app/blueprints/sql_i.py)

```
1  # ZRANITELNÝ KÓD (Slepování řetězců)
2  # Útočník může zadat: ' OR '1'='1
3  raw_sql = f"SELECT * FROM bds.dummy_sql_i WHERE username = '{
    search_input}'"
4  # Přímé spuštění bez ošetření
5  result_proxy = db.session.execute(text(raw_sql))
6
7  # BEZPEČNÝ KÓD (Parametrizovaný dotaz)
8  safe_sql = "SELECT id, username, '*****' as secret_data FROM bds.
    dummy_sql_i WHERE username = :val"
9  # Použití parametrů přes SQLAlchemy
10 result_proxy = db.session.execute(text(safe_sql), {'val': search_input
    })
```

Význam Prepared Statements Použití parametrizovaných dotazů (Prepared Statements) spočívá v oddělení interpretace SQL příkazu od dat. Při použití `text(sql, params)` databáze nejprve zkompileje strukturu dotazu a teprve poté do něj bezpečně vloží uživatelská data. Tím se zabrání tomu, aby útočník pomocí speciálních znaků (např. `' OR '1'='1`) změnil logiku dotazu a získal neautorizovaný přístup k datům.

6.2 Šifrování dat

Telefonní čísla zákazníků jsou šifrována pomocí `pgp_sym_encrypt` (rozšíření `pgcrypto`) přímo v databázi. V tabulce `customer_profile` je uložen binární sloupec `phone_number_enc`. Data jsou čitelná pouze pokud aplikace poskytne správný dešifrovací klíč v SQL dotazu (pomocí `func.pgp_sym_decrypt`).

7 Zálohování a obnova

7.1 Zálohování a Obnova (Backup & Restore)

Pro zajištění bezpečnosti dat jsou implementovány Bash skripty. Skript `backup_db.sh` vytvoří binární dump databáze a odešle jej do AWS S3. Pro obnovu slouží skript `restore_db.sh`, který stáhne vybranou zálohu a provede `pg_restore`.

Zálohovací skript (scripts/backup_db.sh)

```
1 TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)
2 BACKUP_FILE="$BACKUP_DIR/backup_${TIMESTAMP}.dump"
3
4 # 1. Lokální dump pomocí pg_dump (formát custom -Fc)
5 pg_dump -h "$DB_HOST" ... -F c -f "$BACKUP_FILE" "$DB_NAME"
6
7 # 2. Upload na S3 (Cloud Object Storage)
8 aws s3 cp "$BACKUP_FILE" "$S3_BUCKET/"
```

8 Instalace a konfigurace

8.1 Konfigurace (.env)

Aplikace načítá konfiguraci z proměnných prostředí.

Konfigurace (.env)

```
1 DB_HOST=localhost
2 DB_USER=postgres
3 SECRET_KEY=tajny_klic_flask
```

9 Možná budoucí rozšíření

Projekt je navržen modulárně a umožňuje další rozšiřování funkčnosti:

- **REST API:** Implementace API endpointů (např. pomocí Flask-RESTful) pro možnost napojení mobilní aplikace.
- **Dockerizace:** Vytvoření Dockerfile a `docker-compose.yml` pro snazší nasazení celého stacku (App + DB) do kontejnerů.

10 Závěr

Vytvořená aplikace úspěšně splňuje všechny požadavky kladené na projekt. Poskytuje bezpečné a uživatelsky přívětivé rozhraní pro správu e-shopu, využívá pokročilé databázové funkce (triggery, transakce), zajišťuje bezpečnost (Argon2, SQLi prevence) a disponuje automatizovaným zálohováním.